ways of answering those needs. You'll find solutions that change as both the developers and the users become more experienced with the problems they're trying to solve. You'll find a framework that delivers working software early in the development cycle. This software might be rough around the edges, but it lets the users start to get a glimpse of what you'll be delivering.

In this way, Rails encourages customer collaboration. When customers see how quickly a Rails project can respond to change, they start to trust that the team can deliver what's required, not just what's been requested. Confrontations are replaced by "What if?" sessions.

The agile way of working that Rails encourages is tied to the idea of being able to respond to change. The strong, almost obsessive, way that Rails honors the DRY principle means that changes to Rails applications impact a lot less code than the same changes would in other frameworks. And since Rails applications are written in Ruby, where concepts can be expressed accurately and concisely, changes tend to be localized and easy to write. The deep emphasis on both unit and system testing, along with support for test fixtures and stubs during testing, gives developers the safety net they need when making those changes. With a good set of tests in place, changes are less nerve-racking.

Rather than constantly trying to link Rails processes to agile principles, we've decided to let the framework speak for itself. As you read through the tutorial chapters, try to imagine yourself developing web applications this way, working alongside your customers and jointly determining priorities and solutions to problems. Then, as you read the more advanced concepts that follow in Part III, see how the underlying structure of Rails can enable you to meet your customers' needs faster and with less ceremony.

One last point about agility and Rails—although it's probably unprofessional to mention this—think how much fun the coding will be!